



Full-Stack Software Development

Lab 8: React Context, Front-end security

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Contents

1 INTRODUCTION	3
2 GETTING STARTED	4
3 CONNECTING TO A SECURE API	6
3.1 FETCHING REAL DATA	6
3.2 PROVIDING COOKIES FROM A SERVER COMPONENT	7
3.3 TEST & OBSERVE: THE “401 UNAUTHORIZED” ERROR	7
4 IMPLEMENT FRONT-END AUTHENTICATION	9
4.1 USERSERVICE: AUTHENTICATE IN THE BACK-END	9
4.2 BUILDING GLOBAL STATE	9
4.3 PROVIDING THE CONTEXT TO ROOTLAYOUT	10
4.4 CONNECTING THE USERLOGINFORM	11
4.5 CONSUMING AUTHCONTEXT IN HEADER	11
4.6 AUTHCONTEXT REVISITED: IMPLEMENTING LOGOUT	12
5 FINAL TESTING & NEXT STEPS	13
5.1 TESTING THE FULL AUTHENTICATION FLOW	13
5.2 NEXT STEPS: IMPLEMENTING AUTHORIZATION	13

1 Introduction

In this lab, we will refactor the frontend-lecturers into a secure front-end application.

We will implement a complete authentication flow (login/logout) that utilizes the back-end's HttpOnly secure cookie strategy.



The concepts covered include:

- ✓ Global state management with React Context.
- ✓ Working with the secure cookie flow.
- ✓ Building a dynamic UI that reacts to the user state (Login/Logout buttons).



For an in-depth explanation of the concepts we will be applying in this lab, please consult these three presentations:

- 01 - React Context
- 02 - Frontend Security

2 Getting started

To begin this lab, please accept the following GitHub assignment:

<https://classroom.github.com/a/OULmrShv>

Once you have done this, you will be provided with a repository containing the starter code. Clone it to a local directory and open it in VSCode.

Start the back-end application

This lab requires the backend-lecturers application you implemented in **lab 7** to be running:

1. Open the backend project in IntelliJ.
2. To allow incoming requests from a front-end running on <http://localhost:8080>, you will need to add extra CORS configuration. Create a new file “WebConfig.java” in the folder “configuration” and add the following content:

```
package be.ucll.se.courses.backend.configuration;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import
org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:8080")
            .allowedMethods("GET", "POST", "PUT", "DELETE",
"OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

3. Run the CoursesBackendApplication to start the server.
4. The back-end API will be available at <http://localhost:3000>.

Prepare the front-end application

1. Open the cloned *lab-08-fe-security* project in VSCode.
2. Install all required dependencies by running the following command in the terminal:

```
npm install
```

Connect the front-end to the back-end

Our front-end application needs to know the URL of our back-end API. We will provide this using an environment variable.

1. In the root of your frontend-lecturers project, create a new file named: *.env*
2. Add the following content to this file:

```
NEXT_PUBLIC_API_URL=http://localhost:3000
```

3. Start your Next.js development server:

```
npm run dev
```

4. Open <http://localhost:8080> in your browser. You should see the application running with hard-coded data.

3 Connecting to a secure API

Currently, our application displays hard-coded data. In this section, we will modify our `LecturerService` to fetch real data from our secure back-end API.

3.1 Fetching real data

Open `services/LecturerService.ts`. Your first task is to replace the hard-coded lecturer array with a live data fetch. The `getAllLecturers` function will be called by `lecturers/page.tsx`, which is a Server Component. As explained in the slides, fetching from a server component requires a special setup: we must manually read the cookie on the server and send it in an Authorization header.

This special setup is required because the `fetch` call from a server component runs on the Next.js server, not in the user's browser. The server, therefore, must act as a proxy: it first reads the `authToken` cookie that the user's browser sent to it and then manually attaches that token as an `Authorization: Bearer ${token}` header on the new `fetch` request it makes to the back-end API.



Modify the `getAllLecturers` to fulfill these new requirements.:

- ✓ Refactor the `getAllLecturers` function to accept the user's session information (the cookies) as an argument.
- ✓ The function must use this information to perform an authenticated fetch call to the back-end API.
- ✓ The function must ensure that it always retrieves the latest data from the server, not a cached version.
- ✓ The function must correctly handle and report any API errors (like a 401) by throwing an error

3.2 Providing cookies from a server component

Now, open `app/lecturers/page.tsx`. This server component already contains a `getData` helper function that calls your `LecturerService`. The `LecturerService.getAllLecturers` function (which you just refactored) now has a new requirement: it needs the user's cookies to be passed as an argument. Update the `getData` function to fulfill this new requirement.

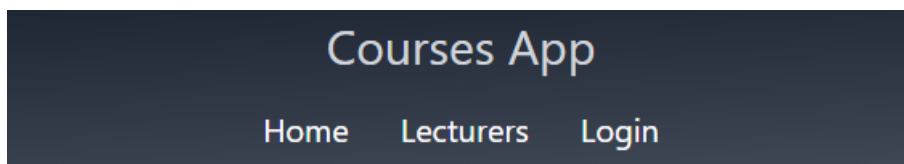


- ✓ Modify the `getData` function so that it can access the user's `cookieStore`.
- ✓ Ensure that this `cookieStore` is correctly passed as an argument to the `LecturerService.getAllLecturers` function.

3.3 Test & observe: The “401 Unauthorized” error

Now that both `LecturerService` and `lecturers/page.tsx` have been refactored to perform a server-side data fetch, it's time to test the result.

Run your application and navigate to the `/lecturers` page. Because we are not logged in, our Next.js server does not have an `authToken` cookie to forward to the API. The back-end will refuse the request, our `LecturerService` will catch this and throw the 401 error, and the `page.tsx` should correctly render the error message.



Lecturers

You are not authorized to view this content. Please login first.

Also, open your browser's Developer Tools (F12) and look at the network tab. You will not see a fetch request to `http://localhost:8080/lecturers`. This is an important observation: it confirms the data fetch is happening server-side and not in the client.



- ✓ Verify that the “401 Unauthorized” error message, thrown by the *LecturerService*, is correctly displayed on the page.
- ✓ Observe in the browser's Network Tools that the API call (*/lecturers*) is happening server-side and is **not** visible in the client

4 Implement front-end authentication

In the previous section, we encountered a “401 Unauthorized” error. This is the expected result of accessing a protected API route without being logged in.

To fix this, we must now implement the complete client-side authentication flow as discussed. We will start by enabling our *UserService* to handle secure cookies, then create a global *AuthContext* to manage the user's state and finally connect our components to this context.

4.1 UserService: authenticate in the back-end

Open *services/UserService.ts*. Currently, this file does not call the back-end API. It simulates login by checking against a hard-coded user. Replace this simulated behavior with real network requests.



- ✓ Refactor the simulated login function to authenticate.
- ✓ This *authenticate* function must perform a real *fetch* call to the */users/login* API endpoint.
- ✓ The fetch call must be correctly configured to receive and store the HttpOnly *authToken* cookie sent by the back-end

4.2 Building global state

In the previous step, our *authenticate* function successfully logs in and receives the user's data (like *fullname* and *role*) as a JSON object. However, this data is currently “stuck” inside the *UserLoginForm* component. Other components, like the *Header*, have no way of knowing who is logged in. To solve this, we could pass the user data as props from the *layout.tsx* file down to the *UserLoginForm* and *Header* components. However, this would be a classic case of “prop drilling”. As we saw in the *React Context* presentation, this is poor practice because it forces intermediate components to accept and pass down props that they don't actually

use themselves, making our code harder to maintain. A much cleaner solution, which keeps our components decoupled, is to use React Context. We will create a global *AuthContext* to make the user state available to any component that needs it, without prop drilling.

A starter file has been provided for you at *context/AuthContext.tsx*. This file already contains the React Context boilerplate. Your goal is to implement the state management logic that makes this context functional:



- ✓ Implement persistence: The provider needs to check *sessionStorage* when the app loads (inside a *useEffect*) to restore the user's session if one already exists.
- ✓ Implement the *login* function: This function must be updated to:
 - Set the user state with the *userData* it receives.
 - Persist this *userData* to *sessionStorage* so it can be restored on a page refresh.
- ✓ You don't need to implement the *logout* function yet.

4.3 Providing the Context to RootLayout

Our *AuthProvider* component is now implemented, but no component can access its state yet. Just like the *ThemeContext* example, a context is useless until it is “provided” to the component tree.



- ✓ Modify the *app/layout.tsx* file to import your *AuthProvider*.
- ✓ Ensure the *AuthProvider* is placed in the component tree so that all child components (like the *Header* and *UserLoginForm*) can consume its state.

4.4 Connecting the UserLoginForm

With our *AuthProvider* now available in *layout.tsx/layout.tsx*, the *UserLoginForm* component is the next logical piece to connect. This component is the “producer” of our authentication state. Its primary responsibility is to capture user credentials, send them to the *UserService*, and update the global *AuthContext* with the authenticated user's data.



- ✓ Modify the *UserLoginForm* to consume *AuthContext* and gain access to the *login* function.
- ✓ Update the *handleSubmit* function so that after a successful authentication, it calls the login function from the context, passing it the *loggedInUser* data.

4.5 Consuming AuthContext in Header

Now that our *UserLoginForm* correctly “produces” or “sets” the global *AuthContext* state upon login, we can modify our Header to “consume” that state. Open *components/header.tsx*. This component is currently static and always displays a “Login” link. Make this component responsive to the user’s authentication status. It needs to read the user object from *AuthContext* and render different content based on whether the user is logged in or not.



- ✓ Refactor the *Header* component to consume the *AuthContext*.
- ✓ The component must conditionally render its content:
 - If a user is present in the context, it should display a “Welcome” message (using the user's fullname) and a “Logout” button.
 - If the *user* is *null*, it should only display the “Login” link.
- ✓ The “Logout” link must be wired up to call the logout function from the *AuthContext* when clicked.

4.6 AuthContext revisited: Implementing logout

Our authentication flow is now almost fully functional. Users can log in, and the *Header* correctly updates to show their name. The “Logout” button is also connected, but it currently calls an empty logout function in our *AuthContext*. A complete and secure logout is a two-part process:

1. Server-side: We must tell the back-end to invalidate and delete the secure *HttpOnly* cookie.
2. Client-side: We must clear our local user state (*useState* and *sessionStorage*) to update the UI.



- ✓ First, the logout function in *UserService* must be implemented.
- ✓ This function must communicate with the back-end's */users/logout* API endpoint and, in doing so, ensure that the *authCookie* is included.
- ✓ The logout function in *AuthContext* must call the *UserService.logout* function to handle the server-side session.
- ✓ It must then always clear the *local session state*, both from React's *useState* and from *sessionStorage*, to ensure the UI updates correctly.

5 Final testing & Next steps

5.1 Testing the full authentication flow

With the logout function now fully implemented, our authentication flow is complete and can be tested now:

1. Log in to the application and check that the Header updates to the “Welcome” message.
2. Navigate to the `/lecturers` page. Verify that you no longer get a 401 error and that the data is correctly fetched (proving your `authToken` cookie was sent server-side).
3. Click the “Logout” button in the Header. The header should immediately update to show the “Login” link.
4. Try to navigate back to the `/lecturers` page. You should now see the “401 Unauthorized” error again, confirming your server-side session cookie was successfully destroyed.

5.2 Next steps: implementing authorization

Our application is now authenticated (we know who the user is). The next logical step is authorization (knowing what the user is allowed to do). The user object in your *AuthContext* contains the user’s role (e.g., “admin” or “lecturer”). You can now use this information to implement role-based features in the UI.



Optional challenge:

- ✓ Modify a component (e.g., *CourseOverviewTable*) to consume *AuthContext* and use the *user.role* to conditionally render a new UI element (like a “Schedule” button) that is only visible for the “admin” role.